• TECHNICAL REPORT — CONFIDENTIAL

AcademyBase

Full System Technical Overview & Architecture Review

A comprehensive engineering analysis of the AcademyBase SaaS platform — covering architecture, database design, module structure, performance profile, identified weaknesses, and a prioritized improvement roadmap.

VERSION	DATE	STACK	STATUS
1.0.0	March 2026	Next.js 16 + Supabase	Production

- Next.js 16.1 (App Router)
- React 19.2 + TypeScript 5
- Supabase 2.97 (PostgreSQL)
- Tailwind CSS 4
- Row-Level Security (RLS)
- Arabic RTL — Sports Management

TABLE OF CONTENTS

1 Executive Summary

2 System Architecture

3 Technology Stack

4 Database Architecture

5 Core Modules

6 Financial System

7 KPI & Analytics

8 Authentication & Security

9 Codebase Structure

10 Performance Analysis

11 Product Weaknesses

12 Improvement Opportunities

13 Future Roadmap

Executive Summary

AcademyBase is a full-stack SaaS platform purpose-built for Arabic-speaking sports academies. It provides a complete operational system covering everything from player registration to financial reporting.

13	28	33	~13.5k
DATABASE TABLES	REACT PAGES	SQL MIGRATIONS	LINES OF CODE

What is AcademyBase?

AcademyBase serves as the operating system for football academies. It replaces fragmented tools — spreadsheets, WhatsApp group notes, manual ledgers — with a unified, role-aware platform accessible from any device.

The platform is fully localized in Arabic with an RTL layout, targeting academy owners and managers in Gulf and Arab markets. It handles the complete operational lifecycle: onboarding new players, tracking attendance session-by-session, managing subscriptions across billing cycles, recording all financial flows, and surfacing actionable insights through dashboards and AI-driven analysis.

Core Value Proposition

OPERATIONAL CLARITY

Real-time view of all active players, their subscription status, upcoming renewals, and attendance streaks — eliminating guesswork from daily academy management.

FINANCIAL TRANSPARENCY

Auto-synchronized revenue from payments, configurable expense categories, staff salary tracking, and branch-level P&L — all in one ledger with override controls.

MULTI-ROLE TEAM ACCESS

Owner, partner, branch manager, and admin staff roles with distinct data access controls enforced at both UI and database levels via PostgreSQL RLS policies.

AI-READY INSIGHTS

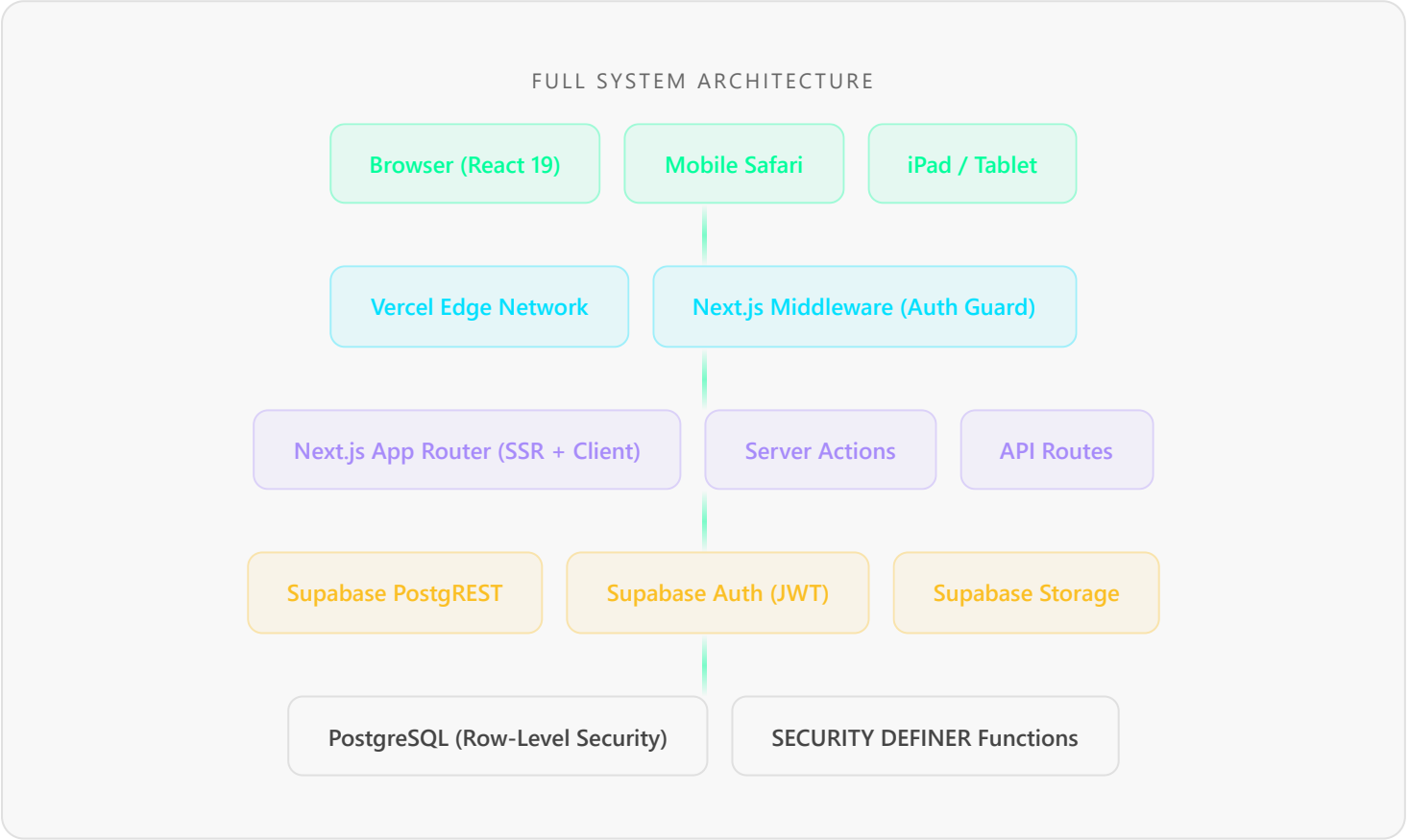
A pure-computation insight engine surfaces critical signals: renewal rate drops, revenue anomalies, expiring subscriptions, and attendance red flags — prioritized by severity.

Current Capabilities

- Player CRUD with subscription mode support (session-based and monthly billing)
- Multi-branch management with configurable schedules, pricing, and session calendars
- Attendance tracking with session-period awareness and optimistic UI updates
- Financial ledger with auto-sync from payments, staff salaries, and field costs
- Role-based team collaboration with invitation-based onboarding
- Insights engine with 5 signal types and severity classification
- Analytics with trend charts, renewal rates, age distribution, and seasonality
- Monthly training calendar with match, cancellation, and special event tracking
- Responsive mobile-first RTL design with dark theme

System Architecture

AcademyBase uses a modern serverless-first architecture with Next.js App Router and Supabase as the sole backend. There is no custom API server — all data access goes directly through Supabase's PostgREST API, secured by Row-Level Security policies.



Frontend Architecture

The frontend is a React 19 single-page application rendered within Next.js App Router. Protected pages are client components (`"use client"`) that fetch data on mount via `useEffect` + Supabase browser client. Public auth pages (login, register, invite accept) are server components leveraging Next.js SSR.

The layout hierarchy is: Root Layout → App Layout (sidebar + mobile header state) → Page. The App Layout is a client component that manages the sidebar drawer state for responsive behavior.

Backend Architecture

There is no custom API server. Supabase's auto-generated PostgREST API serves all data requests. Authentication uses Supabase Auth (JWT-based). Role and access control is enforced at the database layer via PostgreSQL Row-Level Security — the application layer simply reflects the data Supabase returns.

Server Actions (Next.js) are used for critical write operations that require the service-role key: accepting invitations (`acceptInvitationAction`), creating academies (`setupAcademyAction`), and invited user setup (`setupInvitedUserAction`).

Data Flow Pattern

1. **Auth Check (Middleware)**
   Every request through `middleware.ts` verifies a valid Supabase session. Unauthenticated users are redirected to `/login` . Server Actions bypass this redirect to prevent client-side errors.

2. **Academy ID Resolution (Cached)**
   `resolveAcademyId()` reads `profiles.academy_id` for the current user. The result is cached as a module-level Promise — all concurrent callers share a single in-flight query. Only 1 database call per session.

3. **Parallel Data Fetch**
   Pages execute `Promise.all([...libFunctions])` where each lib function uses the cached academy_id. This achieves parallel DB queries with a single resolved identifier.

4. **RLS Enforcement (Database)**
   Supabase applies RLS policies on every query. SECURITY DEFINER helper functions resolve user context without recursive policy evaluation. Non-authorized users receive empty result sets, not errors.

5. **State + Render**
   Data lands in React state. `useMemo` computes derived metrics (renewal rate, KPIs). Charts render via Recharts (tree-shaken). Optimistic UI updates for attendance toggling.

Technology Stack

AcademyBase uses a deliberately minimal, modern stack chosen for developer velocity, deployment simplicity, and production-grade reliability.

LAYER	TECHNOLOGY	VERSION	ROLE	WHY CHOSEN
Framework	`Next.js`	16.1.6	Full-stack React	App Router, Server Actions, SSR, Vercel edge deployment
UI Library	`React`	19.2.3	Component rendering	Latest concurrent features, stable for production
Language	`TypeScript`	5.x	Type safety	Strict mode enforced, eliminates runtime type errors
Styling	`Tailwind CSS`	4.x	Utility CSS	RTL-compatible, zero-config dark mode, fast iteration
Backend/DB	`Supabase`	2.97.0	Database + Auth + API	PostgreSQL + PostgREST auto-API + JWT Auth + RLS in one service
Charts	`Recharts`	3.7.0	Data visualization	Composable React-native charts; tree-shaken via next.config
Build	`Turbopack`	Built-in	Dev bundler	10–15× faster dev server HMR vs. Webpack
Database	`PostgreSQL`	15.x (Supabase)	Relational DB	ACID, RLS, JSONB, full SQL — hosted by Supabase

Architectural Philosophy

ZERO BACKEND SERVERS

No Express, no Django, no custom API. All data operations go through Supabase PostgREST. This eliminates an entire infrastructure tier and reduces operational overhead to near zero.

ZERO STATE MANAGEMENT LIBRARY

No Redux, no Zustand, no Jotai. Plain React hooks (`useState` , `useMemo`) with Supabase as the single

source of truth. Eliminates boilerplate and synchronization bugs.

ZERO LOCALSTORAGE

All user data persists in Supabase only. Session-scoped data uses module-level memory (cleared on logout). This prevents stale data bugs across login sessions and is more secure.

SECURITY AT THE DATABASE

Row-Level Security enforces authorization at the data layer. The frontend doesn't need to implement access control — RLS returns only the rows each user is permitted to see, regardless of how queries are made.

✓ This stack combination enables a single developer to build and operate a production SaaS. Supabase handles authentication, database hosting, backups, and the REST API. Next.js + Vercel handle global CDN, SSR, and serverless compute. The total infrastructure cost at moderate scale is under $50/month.

Database Architecture

The database is a PostgreSQL instance managed by Supabase with 13 tables, 33 migration files (~3,900 SQL lines), and a carefully designed non-recursive Row-Level Security system.

Table Schema

TABLE	PRIMARY KEY	FOREIGN KEYS	PURPOSE	ROW COUNT (EST.)
`academies`	uuid	owner_id → auth.users	Academy organization record	Low (1 per academy)
`profiles`	user_id	academy_id → academies	CANONICAL: user → academy mapping	Low (1 per user)
`academy_members`	uuid	academy_id, user_id	Role-based team membership	Low (team size)
`branches`	uuid	academy_id	Training locations with pricing	Low (2–10)
`players`	uuid	academy_id, branch_id	Subscriber records	Medium (50–1000)
`payments`	uuid	academy_id, player_id, branch_id	Subscription payment history	High (grows monthly)
`staff`	uuid	academy_id	Coach/admin records with salary	Low (5–50)
`finance_tx`	uuid	academy_id	Income/expense ledger	High (auto-generated)
`attendance`	uuid	academy_id, player_id, branch_id	Per-session attendance record	Very High (daily)
`calendar_events`	uuid	academy_id, branch_id	Training/match/event calendar	Medium (monthly)

TABLE	PRIMARY KEY	FOREIGN KEYS	PURPOSE	ROW COUNT (EST.)
`invitations`	uuid	academy_id, invited_by	Team invite tokens	Very Low
`sessions`	uuid	branch_id	Per-session financial costs	Medium
`subscription_events`	uuid	academy_id, player_id	Subscription lifecycle audit log	High

Key Relationships

```sql
-- Core entity graph
academies
  └── profiles          (academy_id) — 1 user profile per academy
  └── academy_members   (academy_id) — team membership + roles
  └── branches          (academy_id) — training locations
        └── players     (branch_id)  — subscribers
              └── payments            (player_id) — billing history
              └── attendance          (player_id) — daily presence
              └── subscription_events (player_id) — lifecycle audit
  └── staff             (academy_id) — coaches + admins
  └── finance_tx        (academy_id) — income/expense ledger
  └── calendar_events   (branch_id)  — training calendar
  └── sessions          (branch_id)  — per-session cost records
```

Row-Level Security Design

The most technically complex part of the database is the non-recursive RLS design. A naive approach of referencing `academy_members` inside policies creates a circular dependency (error 42P17: infinite recursion detected). AcademyBase solves this with a canonical "source of truth" pattern and SECURITY DEFINER helper functions.

> ⚠ **The Recursion Problem:** If `branches.SELECT` policy queries `academy_members`, and `academy_members.SELECT` policy queries `academies`, and `academies.SELECT` queries `academy_members` again — PostgreSQL enters infinite recursion. This is a common Supabase pitfall.

SOLUTION: SECURITY DEFINER HELPERS

```sql
-- Helper runs as postgres (BYPASSRLS) — breaks recursion
CREATE OR REPLACE FUNCTION public.my_academy_id()
RETURNS uuid
LANGUAGE sql STABLE SECURITY DEFINER
AS $$
  SELECT academy_id FROM profiles
```

```
  WHERE user_id = auth.uid();
$$;

-- RLS policy on branches uses helper, NOT inline subquery
CREATE POLICY "branches_select" ON branches
  FOR SELECT USING (academy_id = my_academy_id());
```

TABLE	SELECT POLICY	WRITE POLICY
academies	owner OR member of academy	Owner only
profiles	user_id = auth.uid()	Self only
branches, players, payments	my_academy_id()	Owner + admin
finance_tx	Owner + admin	Owner + admin
attendance, calendar_events	All academy members	Owner + partner + branch_manager
invitations	Owner + invitee	Owner insert/delete; invitee accept

Migration Strategy

33 sequential migration files form the complete schema history. Each file is idempotent where possible (DROP IF EXISTS + CREATE OR REPLACE). Migration 14 is the critical fix migration that resolves the RLS recursion bug and can be safely re-run on any environment.

Key Indexes

- `UNIQUE(academy_id, user_id)` on academy_members — prevents duplicate membership
- `UNIQUE(academy_id, player_id, date)` on attendance — upsert conflict target
- `UNIQUE(academy_id, auto_key)` on finance_tx — idempotent auto-sync entries
- `UNIQUE(branch_id, date)` on sessions — per-day field cost record
- `INDEX on payments(player_id, date)` — renewal rate queries scan by player

Core Modules

AcademyBase consists of 8 core functional modules, each with its own route, data layer, and business logic. All modules share the session-scoped academy_id cache and parallel data fetching pattern.

Dashboard

The central command center. Displays 8 KPI tiles (active players, revenue, renewal rate, upcoming renewals, expenses, net profit, staff count, branch count). Shows the top 3 AI insights, renewal rate chart, and finance trend. Data loads in a single `Promise.all` with 6 parallel Supabase queries. Renewal rate uses an `O(n)` useMemo map instead of naive `O(n×m)` filter. Finance KPIs are hidden for staff roles.

~600 lines | Recharts | useMemo | Role-aware

Players

Full player management with add/edit/delete/renew/extend/pause/resume operations. Filter tabs for All, Active, Ending Soon (7 days), and Expired. Status computed from payment period history (not just player.end_date — all periods are checked). Mobile shows card list, desktop shows data grid. Subscription modes: session-based (حصص) or monthly (شهري). Subscription period history preserved across renewals.

~1000 lines | Responsive | CRUD Modals | Period Tracking

Attendance

Sub-route under Players (`/players/attendance`). Branch tabs + date chips allow navigation by month. Session dates are computed from `branches.days[]` (no DB queries for calendar generation). Three-state per player: active (checkable), not_subscribed (locked), expired (locked). Optimistic UI toggles update local state immediately, rolling back on error. Players/branches cache across month changes; only attendance re-fetches.

Optimistic UI | Period-aware | Cached load

Finance

The most complex module (~1,500 lines). Auto-sync aggregates payments by month/branch into revenue entries and staff salaries into expense entries. Manual entries supported. Grouped view shows categories with sub-items. Users can suppress auto entries (sets source="suppressed") to prevent regeneration. Conflict key (academy_id, auto_key) ensures idempotent upserts. Sort by date/amount. Month selector persists across navigations.

~1500 lines | Auto-sync | Upsert | Suppression

Branches

Branch management with configurable schedule (days of week), pricing (monthly or session-based), field rental cost tracking, and subscription mode. Branches drive attendance calendars and payment calculations. branch_ids[] array on staff records controls which staff are assigned to which branches for filtered views.

Schedule Config | Pricing | Field Rental

Statistics

Analytics page with 5 charts: finance trend AreaChart (3/6/12 month range), renewal rate BarChart per month, age distribution BarChart with branch filter, seasonality BarChart (best months for new subscriptions), and attendance rate by branch. Attendance re-fetches on range change; players/branches cached. All charts built with Recharts.

~800 lines | 5 Charts | Range Selector

Calendar

Monthly grid calendar with training session generation from branch schedules (pure computation, no DB). Event types: training, match, canceled, special_event. `deduct_sessions` flag tracks session consumption. Canceled days show red dot indicator. Branch filter. Role-based edit access (owner/partner/branch_manager can edit; admin_staff view-only). Modal for create/edit/delete.

~800 lines | RTL Grid | Role-gated

Insights

AI insights page powered by a pure-computation engine (`computeInsights()` in `src/lib/insights.ts`). Five signal types: renewal rate drop, revenue drop, upcoming expirations, consecutive absences, and low branch attendance. Severity classification (critical/warning/info). Filter by severity and scope. Dashboard shows top 3 insights. No additional DB calls — uses already-loaded data.

~396 lines | Pure Computation | Severity Filter

Financial System

The financial engine is the most sophisticated module. It combines auto-generated entries from operational data with manual entries, giving academy owners a real-time view of their financial position.

Transaction Categories

REVENUE (إيراد)

- Subscription payments (auto-synced from payments table)
- Aggregated by month + branch
- Separated by new vs. renewal payments
- Manual revenue entries (sponsorships, events)

EXPENSES (مصروف)

- Staff monthly salaries (auto-synced from staff records)
- Field rental costs (per-session from sessions table)
- Manual expense entries (equipment, utilities)
- Staff attendance adjustments (bonuses/deductions)

Auto-Sync Engine

When a user opens the finance page for a given month, the system automatically generates transactions from operational data via `upsertAutoFinanceTx()`. This uses `INSERT ... ON CONFLICT (academy_id, auto_key) DO UPDATE` — making every sync idempotent. Existing manual entries are never overwritten.

```typescript
// Auto-sync generates entries like:
{
  auto_key: "payment_2026-03_branch_uuid",  // unique per month+branch
  type: "إيراد",
  amount: 1250,                             // sum of all branch payments
  source: "auto",
  month: "2026-03"
}

{
  auto_key: "salary_2026-03_staff_uuid",    // unique per month+staff
  type: "مصروف",
  amount: 800,                              // monthly_salary from staff record
  source: "auto",
  month: "2026-03"
}
```

Suppression System

Instead of deleting auto-generated entries (which would cause them to regenerate on next sync), users can "suppress" entries. This sets `source = "suppressed"` and `amount = 0`. On subsequent syncs, the conflict resolution skips suppressed entries, preserving the user's intent.

Session Deductions

The sessions table tracks field rental cost per training session. When a calendar event is marked with `deduct_sessions = true`, it contributes to the branch's cost calculation for that month. This enables per-session financial tracking alongside monthly aggregates.

Salary Calculation

Staff salaries are fixed monthly amounts stored in `staff.monthly_salary` (non-negative constraint enforced by DB). Staff attendance adjustments (bonuses, deductions tracked in `staff_attendance` table) can modify the effective monthly amount. The auto-sync reads both and computes net salary per staff member.

OPERATION	SOURCE	AUTO-KEY PATTERN	EDITABLE?
Subscription revenue	payments aggregation	payment_{month}_{branch_id}	Suppressible
Staff salary	staff.monthly_salary	salary_{month}_{staff_id}	Suppressible
Field rental	sessions.field_cost	rent_{month}_{branch_id}	Suppressible
Manual entry	User input	null	Full CRUD
Salary adjustment	staff_attendance	adj_{month}_{staff_id}	Suppressible

KPI & Analytics

The analytics system combines dashboard KPIs, the Statistics page with 5 chart types, and the Insights engine with 5 signal types — giving academy owners a multi-dimensional view of their business health.

Dashboard KPIs

KPI	FORMULA	VISIBILITY
Active Players	Players where end_date ≥ today AND not paused	All roles
Expiring Soon	Players where 0 ≤ (end_date - today) ≤ 7 days	All roles
Monthly Revenue	Sum of payments.amount for current month	Owner + admin
Monthly Expenses	Sum of finance_tx where type="مصروف" for current month	Owner + admin
Net Profit	Revenue - Expenses	Owner + admin
Renewal Rate	(Renewed players / Total active last month) × 100	All roles
Staff Count	Count of is_active staff	All roles
Branch Count	Count of branches	All roles

Renewal Rate Algorithm (Fixed)

The renewal rate calculation required a significant bug fix. The original implementation counted payment rows by `kind` field, which could produce a count of 2 for a single player who made two payments in a month. The corrected algorithm:

- Groups payments by **player_id** using a Map (O(n) instead of O(n×m))
- Finds each player's **first-ever payment date** (earliest date across all payment history)
- Classifies a player as "new" if their first payment falls in the selected month
- Classifies a player as "renew" if they have payment history before the selected month
- Counts distinct players, not payment rows

Insights Engine

The insights engine (`src/lib/insights.ts` , 396 lines) is a pure function — it takes loaded data as input and returns a sorted array of insight objects. This design enables testing without DB access and reuse across dashboard and insights page.

SIGNAL	CRITICAL THRESHOLD	WARNING THRESHOLD	DATA SOURCE
Renewal Rate Drop	≥ 25% drop month-over-month	≥ 10% drop	payments
Revenue Drop	≥ 30% drop month-over-month	≥ 15% drop	payments
Upcoming Expirations	≥ 5 players expiring in 7 days	≥ 2 players	players
Consecutive Absences	≥ 5 absences in a row	≥ 3 absences	attendance
Low Branch Attendance	≥ 35% below academy average	≥ 20% below	attendance

Statistics Charts

FINANCE TREND

AreaChart showing revenue, expenses, and net profit over 3/6/12 months with a range selector.

RENEWAL RATE

BarChart showing new vs. renewed players per month with color-coded bars (green = new, teal = renew).

AGE DISTRIBUTION

BarChart of player age groups with optional branch filter. Reveals target demographic.

SEASONALITY

BarChart of new subscriptions by month of year. Reveals peak enrollment seasons.

ATTENDANCE RATE

BarChart of attendance percentage by branch. Refetches on range change; compares branch performance.

Authentication & Security

Security is enforced at three layers: middleware (route protection), RLS (data access), and service-role server actions (privileged operations). The design follows a zero-trust data model.

Authentication Flow

1 **Sign Up / Sign In**

Standard email/password via Supabase Auth. Returns a JWT token stored in the browser (cookies via Supabase SSR package). Email confirmation supported.

2 **Middleware Route Guard**

`middleware.ts` runs on every request. Reads the Supabase session from cookies. If no valid session, redirects to `/login?next={path}`. Server Actions (POST with `next-action` header) bypass the redirect to prevent client errors.

3 **Academy Registration**

Server Action `setupAcademyAction()` uses service-role key to create: academy record → profile (with academy_id) → academy_members(role="owner"). Atomic within a single Server Action scope.

4 **JWT Token Refresh (iOS Fix)**

iOS Safari's `navigator.locks` API has a 10s timeout. To prevent contention when multiple components race to call `getSession()`, a singleton Supabase client is used in the browser. Only one lock holder at a time — no timeout errors on iPad/iPhone.

Role System

ROLE	ARABIC LABEL	FINANCE ACCESS	STAFF MANAGE	BRANCH EDIT	PLAYER EDIT
owner	المالك	Yes	Yes	Yes	Yes
partner	شريك	Yes	Limited	Yes	Yes
branch_manager	مدير فرع	If granted	No	Own branches	Yes

ROLE	ARABIC LABEL	FINANCE ACCESS	STAFF MANAGE	BRANCH EDIT	PLAYER EDIT
`admin_staff`	اداري	No	No	No	View only

Invitation System

Team members are onboarded via a 32-byte random hex token (256-bit entropy — brute-force infeasible). The invite link is `/invite/{token}` . On acceptance:

- Server validates email match between invite and current user
- Upserts `academy_members` with the specified role
- Updates `profiles.academy_id` (the canonical data store)
- Invite token expires after the configured `expires_at` timestamp

Security Posture Assessment

STRENGTHS

- RLS enforced at DB layer — bypass is impossible without service-role key
- Service-role key used only in Server Actions (never exposed to client)
- 256-bit invite tokens (cryptographically secure)
- No sensitive data in localStorage or client state
- Singleton client prevents iOS Safari lock exploitation

AREAS FOR IMPROVEMENT

- No rate limiting on auth endpoints (Supabase defaults apply)
- No audit log for admin actions (player delete, finance override)
- Branch-scoped staff can theoretically edit players in other branches via direct PostgREST calls
- No 2FA support currently

Codebase Structure

The project follows Next.js App Router conventions with a clear separation between UI pages, reusable components, data access lib, and database migrations.

DIRECTORY TREE

```
academybase/
├── app/                          ← Next.js App Router
│   ├── (app)/                    ← Protected layout group
│   │   ├── layout.tsx            ← Client: sidebar state + mobile header
│   │   ├── dashboard/page.tsx    ← Main KPI dashboard (~600 lines)
│   │   ├── players/
│   │   │   ├── page.tsx          ← Player CRUD (~1000 lines)
│   │   │   ├── attendance/page.tsx ← Attendance tracking
│   │   │   └── layout.tsx        ← Tab bar for players/attendance
│   │   ├── branches/page.tsx     ← Branch management
│   │   ├── finance/
│   │   │   ├── page.tsx          ← Finance ledger (~1500 lines)
│   │   │   └── reports/page.tsx  ← Financial reports
│   │   ├── staff/
│   │   │   ├── page.tsx          ← Staff management
│   │   │   └── attendance/page.tsx ← Staff attendance
│   │   ├── calendar/page.tsx     ← Training calendar (~800 lines)
│   │   ├── statistics/page.tsx   ← Analytics charts (~800 lines)
│   │   ├── insights/page.tsx     ← AI insights
│   │   └── settings/
│   │       ├── page.tsx          ← Academy settings
│   │       └── team/page.tsx     ← Team management (owner only)
│   ├── invite/[token]/page.tsx   ← Server: accept invite
│   ├── login/ & register/        ← Auth pages
│   ├── components/
│   │   ├── Sidebar.tsx           ← RTL sidebar with role visibility
│   │   └── ui/Button.tsx         ← Button component
│   └── layout.tsx                ← Root: dir="rtl", dark theme
├── src/lib/
│   ├── insights.ts               ← Pure insights computation (396 lines)
│   └── supabase/
│       ├── academyId.ts          ← Session-scoped ID cache (CRITICAL)
│       ├── roles.ts              ← Role system + getMembership()
│       ├── browser.ts            ← iOS Safari singleton client
│       ├── server.ts             ← SSR client
│       ├── admin.ts              ← Service-role client
│       ├── players.ts            ← Player CRUD
│       ├── branches.ts           ← Branch CRUD
│       ├── payments.ts           ← Payment CRUD + periods
│       ├── finance.ts            ← Finance TX + auto-sync
│       ├── staff.ts              ← Staff CRUD
│       ├── attendance.ts         ← Attendance CRUD
│       └── calendar.ts           ← Calendar events CRUD
```

```
├── supabase/                  ← 33 migration files
├── middleware.ts              ← Auth guard
└── next.config.ts             ← Recharts tree-shaking
```

Key File Responsibilities

FILE	LINES	RESPONSIBILITY
`app/(app)/finance/page.tsx`	~1500	Finance ledger with auto-sync, manual entry, grouping, suppression
`app/(app)/players/page.tsx`	~1000	Player CRUD with subscription periods, filters, responsive layout
`app/(app)/statistics/page.tsx`	~800	5 analytics charts with range selector and data caching
`app/(app)/calendar/page.tsx`	~800	Monthly calendar with event management and role-based edit
`app/(app)/dashboard/page.tsx`	~600	KPI dashboard with renewal rate chart and top insights
`src/lib/insights.ts`	396	Pure computation engine for 5 insight signal types
`src/lib/supabase/academyId.ts`	137	Session-scoped Promise cache for academy_id resolution
`src/lib/supabase/roles.ts`	117	4-tier role system with getMembership() and permission helpers
`src/lib/supabase/finance.ts`	115	Finance TX CRUD + upsertAutoFinanceTx() with conflict resolution
`middleware.ts`	81	JWT validation + route protection + Server Action bypass

Performance Analysis

The application has several well-implemented performance optimizations, but also contains areas with scaling concerns as data volumes grow.

Existing Optimizations

OPTIMIZATION	IMPACT	FILE
Academy ID module cache	`N profile queries → 1 per session`	academyId.ts
iOS Safari singleton client	`Eliminates 10s lock timeout on mobile`	browser.ts
Promise.all concurrent fetches	`Sequential → parallel DB queries`	All pages
useMemo renewal rate map	`O(n×m) → O(n)`	dashboard/page.tsx
Attendance static load cache	`3 queries cached across month changes`	attendance/page.tsx
Recharts tree-shaking	`~70% bundle reduction (~420KB → ~130KB)`	next.config.ts
Optimistic attendance toggle	`Instant UI response, no loading spinner`	attendance/page.tsx

Performance Score Estimates

Scaling Concerns

⚠ **Full table loads:** Pages like Players and Finance currently fetch all rows for the academy and filter in the browser. With 1000+ players or 5 years of finance history, this becomes a scaling bottleneck. Server-side pagination is not

yet implemented.

⚠ **Attendance table growth:** At 100 players × 3 sessions/week × 52 weeks = 15,600 rows/year. A 5-year academy with 500 players would have ~780,000 attendance rows. The attendance page loads all rows for a month, which is manageable, but the attendance trend query in Statistics loads potentially months of data.

ℹ **Insights not memoized at page level:** `computeInsights()` re-runs on every render cycle where data exists. While it's a pure function and relatively fast, wrapping it in `useMemo` would prevent redundant computation on unrelated state updates.

Database Query Analysis

QUERY	CURRENT	CONCERN	RECOMMENDATION
List all players	Full table scan with RLS	Scale: 1000+ rows	Server-side pagination + search
List all payments	Full table for academy	Grows indefinitely	Range-based fetch (last N months)
Finance TX list	All TX for academy	Years of data	Paginated + date-range filter on server
Attendance by month	Bounded by month	Manageable	Index on (academy_id, date) would help
resolveAcademyId()	1 query/session (cached)	None	Current implementation is optimal

Product Weaknesses

An honest assessment of current limitations across UX, architecture, data design, and performance — to guide the next improvement cycle.

UX Weaknesses

NO GLOBAL SEARCH

There is no cross-module search. Finding a specific player requires navigating to the Players page and using the filter. A cmd+K style command palette would significantly improve navigation speed for power users managing large academies.

NO NOTIFICATION SYSTEM

The insights engine identifies critical signals (expiring players, attendance drops) but there is no proactive notification — not even an in-app notification badge. Users must actively visit the insights page to see alerts.

ARABIC-ONLY INTERFACE

All UI strings are hardcoded in Arabic. There is no i18n layer. Expanding to non-Arabic markets or international coaches on the team would require significant string extraction work.

NO BULK OPERATIONS

Players can only be acted upon one at a time. For academies that want to send renewals to 50 expiring players simultaneously, or mark a class of players as attended after a session, there is no multi-select or bulk action capability.

Architecture Weaknesses

NO SERVER-SIDE PAGINATION

All data is fetched client-side with RLS filtering. As academies grow (500+ players, 3+ years of finance history), initial page loads will degrade. No pagination, virtual scrolling, or server-side filtering is implemented.

NO OPTIMISTIC UPDATES OUTSIDE ATTENDANCE

Only the attendance toggle has optimistic updates. Player edits, payment creation, and finance entries all show loading states with no immediate UI feedback. This creates a noticeable lag feel even when the DB operation succeeds quickly.

CONCURRENT FINANCE SYNC RISK

If two users (owner + admin) open the finance page simultaneously, both trigger `upsertAutoFinanceTx()` for the same month. The UNIQUE conflict resolution handles this correctly, but under high concurrency, there's a theoretical risk of duplicate entries if the conflict key logic changes.

NO REAL-TIME UPDATES

Supabase provides WebSocket-based Realtime subscriptions, but none are used. If two team members are viewing the players list simultaneously and one adds a player, the other won't see the update until they refresh.

Data Design Weaknesses

SUBSCRIPTION PERIOD GAPS

When a player renews, the new `start_date` might create a gap between the old subscription end and the new start. The attendance page handles this correctly (checks all periods), but the Players page status display uses `player.end_date` which is overwritten on each renewal, losing the previous period's end date from the player record itself.

BRANCH_IDS ARRAY ON STAFF

Staff assignment to branches uses a PostgreSQL UUID array (`branch_ids uuid[]`). This is not normalized — it's not enforced by foreign keys, there's no cascading behavior on branch delete, and it can't be efficiently joined with SQL. A junction table would be more robust.

Performance Weaknesses

FINANCE PAGE IS THE HEAVIEST PAGE

At ~1,500 lines, the finance page loads the most data and performs the most complex client-side aggregation. Auto-sync logic that runs on mount makes 5–7 separate upsert calls. As data grows, this page will be the first to show noticeable slowdowns.

INSIGHTS NOT WRAPPED IN USEMEMO

`computeInsights()` iterates all payments, players, and attendance records on every call. Without `useMemo`, this recomputes on every state update in the page — including unrelated state like modal open/close events.

Improvement Opportunities

Prioritized, actionable improvements organized by impact area. Each recommendation includes implementation guidance and expected benefit.

High Priority — Architecture

1 **Server-Side Pagination for Players and Finance**

Implement `range(offset, limit)` Supabase queries with server-side filtering. Use Next.js URL search params for page state. Expected: 80% reduction in initial data load for large academies. Required for scaling beyond 500 players.

2 **Database Indexes on High-Traffic Queries**

Add composite indexes: `(academy_id, date)` on attendance and payments; `(academy_id, end_date)` on players; `(academy_id, month)` on finance_tx. Expected: 5–10× query speed improvement at scale.

3 **Supabase Realtime Subscriptions**

Subscribe to players and attendance table changes via `supabase.channel()` . Enable multi-user collaboration without refresh. Particularly impactful for attendance taking where multiple staff might mark the same session.

Medium Priority — UX

4 **Global Command Palette (Cmd+K)**

Implement a fuzzy-search command palette over players, branches, and navigation. Libraries: cmdk or custom implementation. Transforms player lookup from navigate→filter→find to instant search. Critical for large academies.

5 **In-App Notification System**

Add a notification bell in the header that runs a lightweight version of `computeInsights()` on mount. Show count badge for critical signals. Toast notifications for actions. Enables passive awareness without visiting insights page.

6 **Bulk Player Actions**

Add multi-select checkboxes on the players list. Enable bulk: renew, extend, message, export to CSV. Especially valuable at end-of-month for bulk renewal processing. Reduces 50 individual modal interactions to 1 bulk action.

7 **Optimistic Updates Across All Forms**

Extend the optimistic update pattern from attendance to player CRUD, payments, and branch edits. Update local state immediately on form submit, revert on error. Expected: perceived performance improvement of 200–800ms on all write operations.

Lower Priority — Code Quality

8

Wrap computeInsights() in useMemo

Add `useMemo(() => computeInsights(input), [players, payments, attendance, branches])` in both the dashboard and insights pages. Prevents recomputation on unrelated state updates. Small change, immediate gain.

9

Normalize Staff Branch Assignment

Replace `staff.branch_ids uuid[]` with a junction table `staff_branch_assignments(staff_id, branch_id)`. Enables FK constraints, cascade on branch delete, and efficient branch-based staff queries.

10

Extract Common Page Shell Component

All pages share a pattern: loading state → error state → data display. Extract a `<PageShell loading={} error={}>` component to reduce boilerplate across the 8 main pages. Reduces ~50 lines of repetitive code per page.

Future Roadmap

Strategic feature directions that would transform AcademyBase from an operational tool into an intelligent academy intelligence platform.

Phase 1 — Enhanced Intelligence (Q2 2026)

ADVANCED AI INSIGHTS

Extend the insights engine with LLM-generated natural language explanations. Use Claude API to produce actionable recommendations in Arabic: "3 players from Branch A have missed 4+ consecutive sessions. Consider sending a WhatsApp follow-up this week."

- Connect `computeInsights()` output to Claude API
- Arabic-language response generation
- One-click action triggers from insight cards
- Weekly digest email with top 5 insights

PREDICTIVE RENEWAL ANALYTICS

Use historical payment data to predict which players are likely to churn. Train a simple logistic regression model on: attendance rate, payment timeliness, session completion rate, and absence streaks.

- Churn probability score per player (0–100%)
- Risk segmentation: high/medium/low
- Renewal probability chart on dashboard
- Proactive re-engagement list generation

Phase 2 — Platform Expansion (Q3 2026)

ACADEMY BENCHMARKING

Anonymized cross-academy benchmarking showing how an academy's renewal rate, revenue per player, and attendance rate compare to similar academies by size and geography. Requires opt-in data sharing and aggregate-only API.

- Percentile ranking (top 25% in region)
- Benchmark dashboard card
- Gap analysis vs. top performers
- Best practice recommendations

FINANCIAL FORECASTING

12-month revenue and expense forecast based on historical trends, current subscription mix, and seasonal patterns identified in the Statistics module.

- Monte Carlo simulation with confidence intervals
- Scenario modeling (10% churn vs. 20% growth)
- Break-even analysis per branch
- Staff salary vs. revenue ratio alerts

Phase 3 — Ecosystem (Q4 2026)

PARENT & PLAYER PORTAL

A separate read-only portal where parents can view their child's attendance record, payment history, and upcoming schedule. Reduces inbound WhatsApp messages to academy staff by 60–70% based on industry benchmarks.

WHATSAPP INTEGRATION

Automated WhatsApp messages via WhatsApp Business API: renewal reminders (7 days before expiry), absence alerts (3 consecutive misses), payment confirmations. All triggered from the insights engine signals.

MOBILE NATIVE APP

React Native or Expo wrapper for the attendance and calendar modules. Enables offline-first attendance marking that syncs when connectivity is restored. Critical for field operations with poor internet.

PLAYER PERFORMANCE TRACKING

Add a performance assessment module allowing coaches to score players on configurable attributes (speed, technique, teamwork). Progress tracking over time. Feeds into AI insights about player development trajectories.

Suggested Development Priority Matrix

FEATURE	EFFORT	IMPACT	PRIORITY
Server-side pagination	Medium	High	P0
Notification system	Low	High	P0
Supabase Realtime	Low	Medium	P1
Command palette search	Low	High	P1
Bulk player actions	Medium	High	P1
AI insights (Claude API)	Medium	High	P1
Predictive churn model	High	High	P2
WhatsApp integration	High	High	P2
Parent portal	High	Medium	P2
Academy benchmarking	High	Medium	P3
Financial forecasting	High	Medium	P3

FEATURE	EFFORT	IMPACT	PRIORITY
Mobile native app	Very High	Medium	P3

✓ **Strategic Recommendation:** The current codebase is production-ready and architecturally sound. The highest-ROI next steps are: (1) server-side pagination to unblock scale, (2) notification system to surface existing insights proactively, and (3) Claude API integration to provide natural-language recommendations in Arabic — transforming the existing pure-computation insights engine into a conversational intelligence layer with minimal additional infrastructure.